

Problem sheet 1 for the Lecture course “Algorithms and Computations in Physics” 2025

Ylias Sadki, Werner Krauth

University of Oxford

(Dated: 21 January 2025)

1. **direct-pi** Implement the Algorithm **direct-pi** in Python and confirm that the output approaches $\pi/4$. Modify the program, so that it runs many times, for a given number of trials, say $N = 1000$. Plot a histogram of the output and characterize it completely (use that the variance of a Bernoulli random variable with parameter θ is $\theta(1 - \theta)$).

2. **walker** In Lecture 1, we discussed Walker’s algorithm, a prodigious sampling method of the distribution $\{p_1, \dots, p_K\}$ in $\mathcal{O}(1)$ per sample as $K \rightarrow \infty$.
 - Implement the table construction for this algorithm and explain why at each stage of this construction, one starts with one small stick and one tall stick. What would happen if the construction started with the tall sticks?

 - Implement the sample stage for this algorithm and explain why it is $\mathcal{O}(1)$ per event.

 - Test your implementation in a “Saturday-night” problem of your choice and make sure that it actually picks any element i with probability p_i , to good precision.

3. **Convergence to cumulative distribution** (optional) In Lecture 1, we discussed the Alg. **direct-gamma**, obtained from samples $\text{ran}(0, 1)^\gamma$. We learned that the random variable corresponding to this algorithm, for $\gamma < 1/2$ is very tricky.
 - Take $\gamma = -0.75$. What is the probability distribution $\pi(\mathcal{O})$ of $\mathcal{O} = \text{ran}(0, 1)^\gamma$?

- Write a Python program to test explicitly what you just found out.
- Modify your program, so that it generates N samples of $\text{ran}(0, 1)^\gamma$. Sort these samples so that $x_1 < x_2 < \dots < x_N$ and compute the empirical cumulative distribution function $\hat{F}(x)$, the function that is zero for $x < x_1$, that is $1/N$ for $x_1 < x < x_2$, etc. Compare $\hat{F}$ to the true cumulative distribution function $F(x)$ that you obtained analytically.
- Explicitly check the Glivenko–Cantelli Theorem, stating that

$$\sup_x |\hat{F}_N(x) - F(x)| \rightarrow 0 \quad \text{for } N \rightarrow \infty.$$

- If you have more time, check the Dvoretzky-Kiefer-Wolfowitz inequality, that states:

$$\mathbb{P} \left(\sup_x |\hat{F}_N(x) - F(x)| > \epsilon \right) \leq 2e^{-2N\epsilon^2}$$

This inequality, miraculously, does not depend on the distribution you’re sampling, as long as F is continuous (see Wasserman, Theorem 7.5).

Algorithms and Computations in Physics problem sheet 2, part 1/2
(First of two parts of PS2. Part 2 will be released after lecture 3. Both pieces are due
Saturday 5pm week 3)

Ylias Sadki, Werner Krauth

University of Oxford

(Dated: 28 January 2025)

1. **markov-pi**

- Implement the Algorithm **markov-pi** in Python and confirm that the output approaches $\pi/4$.
- Modify the program, so that it runs several times, for a given large number of trials, say $N = 10,000$. Measure the precision you obtain from the standard deviation of $\hat{\pi} - 3.1415$, where $\hat{\pi}$ is the estimate obtained from a running average. Study this precision as a function of the throwing range δ .
- Familiarize yourself with the Metropolis–Hastings algorithm, then implement the triangle algorithm for the heliport game. In that algorithm, the a priori probability $\mathcal{A}(\mathbf{x}, \mathbf{x}')$ is a constant if $\mathbf{x}'$ is in an equilateral triangle of sides δ , with center $\mathbf{x}$ (Hint: You may use a rejection algorithm to sample a point in the equilateral triangle). Sample $\mathbf{x}'$ and evaluate $\mathcal{A}(\mathbf{x}', \mathbf{x})$.

2. **metropolis** and **factor-metropolis**

- Familiarize yourself with the Algorithm **metropolis** (Algorithm 2.2) that we discussed at the end of Lecture 2 and implement it in Python.
- Confirm that it samples the Boltzmann distribution of the anharmonic oscillator without any approximation.
- Familiarize yourself with the Algorithm **factor-metropolis** (Algorithm 2.3), implement it, and check whether the histogram of x is still that of the anharmonic oscillator. Can you explain why this algorithm is correct?

Algorithms and Computations in Physics problem sheet 2, part 2/2

(Second of two parts of PS2. Both pieces are due Sunday 12pm, 9th Feb)

Ylias Sadki, Werner Krauth

University of Oxford

(Dated: 04 February 2025)

1. lifted-metropolis, zig-zag

- Remind yourself of the results of the **metropolis** exercise.
- Implement the **lifted-metropolis** algorithm to sample the same distribution.
- Implement the event-driven **zig-zag** algorithm to sample the same distribution. Familiarize yourself with the discretization of time that is introduced simply to facilitate the making of the histogram.
- Plot a histogram and check that **metropolis**, **lifted-metropolis** and the event-driven **zig-zag** algorithms sample the same distribution correctly.
- Explain in your own words why the **zig-zag** is correct? Explain the difference between detailed balance and global balance and state which ones of these algorithms satisfy detailed balance, and which only satisfy global balance.

Algorithms and Computations in Physics problem sheet 3, part 1/2
(First of two parts of PS3. Both pieces are due Sunday 12pm week 5 (23rd Feb))

Ylias Sadki, Werner Krauth

University of Oxford

(Dated: 17 February 2025)

1. event-disks

- Remind yourself of the Alg. (**event-disks**), that you can recover on WK's website.
- Implement the Alg. (**event-disks**) so that it steps through equally spaced times and compute the histogram of x -positions (or y -positions), for $N = 4$ disks with $\sigma = 0.15$. Check that you recover the projected density (the “batman”) that we discussed in the lecture. Run larger systems at the same density (fraction of space occupied by the disks) and discuss how the density evolves.
- Compute the histogram of a single velocity $v_x(1)$ of one particle, and check whether it is a Gaussian. Can you connect the parameter of the Gaussian to your choice of velocities?

2. direct-disks, markov-disks

- Implement the Alg. (**direct-disks**) for four particles and for the same parameters as in the previous exercise, and check whether it gives the “batman” figure. Discuss whether the output of this program is exactly the same as for Alg. (**event-disks**), or whether we expect deviations related to the finite size $N = 4$ of the system. Optional: Convince yourself that if you replace the *tabula-rasa* rule by the rule where you only replace the disk that created the overlap (as discussed in the lecture), the result changes.
- Implement the Alg. (**markov-disks**) for the same parameters as before. This should only take 10 minutes (multiplied by the empirical factor π that applies to all human enterprises...). Discuss whether the output of this program is exactly the same as for Alg. (**event-disks**), and for Alg. (**direct-disks**) or whether we expect deviations related to the finite size $N = 4$ of the system.

Algorithms and Computations in Physics problem sheet 3, part 2/2
(*Second of two parts of PS3. Both pieces are due Sunday 12pm week 5 (23rd Feb)*)

Ylias Sadki, Werner Krauth

University of Oxford

(Dated: 18 February 2025)

1. Levy-harmonic.py, Metropolis-harmonic.py, HMC-harmonic.py

- Implement the three algorithms for simulating the harmonic chain that were discussed in Lecture 5 and check that they compute the correct b -independent mean energy $(L^2/N + N - 1)/2$. To do so, simply go on WK's website and look for the pages that are linked to on the main page of the lecture series.
- Explain the Hamiltonian Monte Carlo algorithm in your own words. Explain why the Metropolis step (third line from below, involving the random number) involves the total energy $U + K$ rather than just the potential energy U .
- (optional) Check that the leapfrog algorithm is exactly time reversible.
- (optional) Redo the calculation that leads to the famous Lévy-construction algorithm.

Algorithms and Computations in Physics problem sheet 4, part 1/2

(First of two parts of PS4. Both pieces are due 12pm Sunday 9th March)

Ylias Sadki, Werner Krauth

University of Oxford

(Dated: 25 February 2025)

1. matrix-square

- Review the matrix-squaring procedure discussed in the first part of Lecture 6. Implement **matrix-square**, starting at some high temperature with the Trotter approximation of eq. (6.16), for the harmonic potential $V(x) = \frac{1}{2}x^2$. Check that you recover, at a few different temperatures, the diagonal density matrix of eq. (6.24).
- Use the same matrix-squaring algorithm for the anharmonic potential $V(x) = \frac{1}{2}x^2 + \frac{1}{4}x^4$.

2. naive-harmonic-path

- Implement the simplest possible path-integral simulation **naive-harmonic-path** and check that you recover the analytic results of eq. (6.24) at a few different temperatures.
- Implement the same program for the anharmonic oscillator $V(x) = \frac{1}{2}x^2 + \frac{1}{4}x^4$ and check that you recover the diagonal density matrix obtained by matrix squaring in the previous exercise.
- (Optional:) Discuss how you could use the program **levy-harmonic-path** to devise a high-precision MCMC simulation for the anharmonic oscillator, with the potential $V(x) = \frac{1}{2}x^2 + \frac{1}{4}x^4$.
- (Optional:) Implement this algorithm.

Algorithms and Computations in Physics problem sheet 4, part 2/2

(First of two parts of PS4. Both pieces are due 12pm Sunday 9th March)

Ylias Sadki, Werner Krauth

University of Oxford

(Dated: 04 March 2025)

1. **three-cycles**

- From Lecture 7, review the algorithm to sample random permutations with cycle lengths 1 and 2. Describe it in your own words. Describe in particular the testing stage of the algorithm, and explain what the dictionary is good for.
- Then modify it into a program **three-cycles**, which outputs permutations uniformly sampled among all permutations that can only have cycles of length 1, 2, or 3. Test your algorithm using the dictionary, as for the program **two-cycles**.

2. **Direct-N-bosons**

- From Lecture 7, review the direct-boson sampling algorithm that is available on the course website. Describe it again in your own words.
- Modify the algorithm so that you can run it many times for $N = 100$. Obtain the cycle-weight distribution for different temperatures. Plot this distribution and interpret it.
- In the Lecture 7, it was discussed that Bose–Einstein condensation is a high-temperature phenomenon. Produce some numerical data for different N , in order to illustrate this point.

NB: Note that in the program, the temperature is $T = T^* N^{1/3}$.

NNB: Note that the program is really naive. Beware of overflow in the calculation of the partition function Z .

End of exercises: Thanks for your interest and active participation.